

SS-DAS (Secure Sensor Data Authentication System)

Complete Project Documentation

This document explains the entire SS-DAS architecture, the purpose of every key file in the backend and frontend, the database schemas, and how the advanced security pipeline functions. It is designed to help you understand exactly what the code is doing for your class presentation.

1. System Architecture Overview

The SS-DAS project is an End-to-End Encrypted (E2EE) IoT telemetry pipeline. It is designed to prove that sensor data (like temperature and humidity) can be securely generated on a resource-limited device (ESP32), stored safely in a central database, and only decrypted by authorized users on a web dashboard.

To demonstrate advanced IoT Security, the project implements a **Layered Defense Architecture**:

1. **PUF Challenge-Response (Authentication)**: Prevents cloned devices from connecting.
 2. **AES-256-GCM (Confidentiality)**: Prevents the database/server from reading the data.
 3. **HMAC-SHA256 (Data Integrity)**: Prevents the data from being tampered with.
 4. **ECDSA (Non-Repudiation)**: Proves exactly which device sent the data.
-

2. Database Schema (backend/src/utils/initDb.js)

The project uses **SQLite3** for lightweight, local storage. The database (`ss-das-v2.db`) is structured into four main tables:

users Table

Stores the dashboard administrators.

- `id` : Unique identifier.
- `username` : The login name (e.g., "admin").
- `password_hash` : The bcrypt-hashed password (never stored in plaintext).
- `role` : Permissions level (e.g., "admin").

devices Table

Stores the registered IoT physical devices.

- `device_id` : The unique hardware string (e.g., "SIM_01").
- `public_key` : The ECDSA public key used to verify the device's digital signatures.
- `puf_secret` : A simulated hardware-bound secret used to verify the device's PUF identity.

puf_challenges Table

Stores temporary cryptographic challenges given to devices.

- `device_id` : The device requesting to send data.
- `challenge` : A random 32-byte hex string. The device must hash this string correctly to prove its identity.

sensor_data Table

The main vault for the telemetry data.

- `id` , `sensor_id` , `device_id` , `timestamp` : Standard metadata.
 - `ciphertext` : The AES-encrypted JSON payload containing the temperature.
 - `nonce` : The unique Initialization Vector (IV) ensuring no two ciphertexts are ever the same.
 - `hmac` : The integrity hash.
 - `signature` : The ECDSA digital signature.
-

3. Backend Code Breakdown (`backend/src/`)

Core Server Files

- `app.js` : The central brain of the backend. It initializes the Express.js framework, sets up the `express-rate-limit` (allowing only 100 requests per 15 minutes to prevent DoS attacks), and routes incoming URLs to their specific handler files.
- `utils/db.js` : Establishes the connection to the SQLite database file.
- `utils/initDb.js` : Runs when the server starts to build the database tables if they don't exist yet.
- `utils/initAdmin.js` : Checks if an admin user exists; if not, it automatically creates one (`admin / Security@2025`).

Auth Routing (`backend/src/auth/`)

- `login.js` : Handles `POST /auth/login` . It checks the database for the user, runs `bcrypt.compare` to verify the hashed password, and then generates a **JSON Web Token (JWT)** to keep the user logged in for 2 hours.
- `challenge.js` : Handles `GET /auth/challenge` . When an ESP32 wants to send data, this file generates a random 32-byte hex string (the Challenge) and saves it to the database for that specific device.

Data Routing (`backend/src/data/`)

- `ingest.js` : The most critical security file. It handles `POST /data/ingest` .
 1. It grabs the `puf_response` from the device and calculates what the expected hash *should* be using the database's `puf_secret` . If they don't match, it rejects the payload ("Device clone detected!").
 2. It grabs the ECDSA `signature` and verifies it mathematically using the device's `public_key` .
 3. If both security checks pass, it inserts the encrypted ciphertext into the database.
 - `list_sensors.js` & `sensor_history.js` : Simple endpoints that fetch the stored ciphertexts out of the database and send them to the frontend dashboard.
-

4. Frontend Code Breakdown (`docs/`)

The frontend is a vanilla HTML/JS application built to be hosted on GitHub Pages or directly opened in Chrome.

- `index.html` : The UI skeleton. It contains the Login View, the Dashboard Table View, and the specific Sensor Detail View featuring the Security Badges.
- `app.js` : Controls the flow of the UI (hiding and showing the login vs. dashboard cards depending on if you have a valid JWT token).
- `api.js` : A helper wrapper around the browser's `fetch()` API. It automatically attaches your JWT Authorization token to every request sent to the backend.
- `ui/sensor_view.js` : The interactivity logic for the dashboard. When you click "Decrypt", this file triggers a `prompt()` asking for your password. It then checks the HMAC mathematically, updates the green verification badges, and triggers AES decryption.

Cryptography Engine (docs/crypto.js)

This is where the browser's native **Web Crypto API** is utilized for client-side processing:

- **deriveAESKeyFromPassword()** : Takes the password you typed (`my_secure_password`) and runs it through the **PBKDF2** algorithm 100,000 times to stretch it into a 256-bit AES master key.
 - **verifyHMAC()** : Takes the E2EE AES key and hashes the ciphertext. If it matches the database's HMAC, it proves no one tampered with the data.
 - **decryptAESGCM()** : The final step. It decrypts the ciphertext using the AES key and the `nonce` mathematically, revealing the plaintext JSON `{temperature: 21.5}` .
-

5. Firmware Code Breakdown (firmware/src/main.cpp)

The ESP32 microcontroller acts as a miniature, highly secure server. It uses the embedded C library `mbedtls` to perform all the heavy cryptographic mathematics.

- **Initialization**: It connects to Wi-Fi and initializes the physical DHT11 sensor to read actual temperature and humidity.
- **PUF Subroutine (fetchPUFChallenge())**: It talks to `/auth/challenge` to get a random string, then instantly hashes it with its hidden `PUF_SECRET` .
- **Encryption Subroutine**: It allocates memory, generates a random 12-byte `nonce` , and calls `mbedtls_gcm_crypt_and_tag()` to encrypt the temperature data into unreadable bytes.
- **Integrity & Signatures (computeHMAC() & computeECDSASignature())**: Before sending, it hashes the encrypted data (HMAC), and then uses Elliptic Curve Cryptography (`secp256r1`) to sign a digital certificate proving its identity.
- **Transmission**: It transmits the final package over `HTTPS` (`WiFiClientSecure`) to the backend.